PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

OLEH:

Nama	: Aulia illah
Nim	: 2025573010070
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: M.reza zulman ,S.ST.,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2025

LEMBAR PENGESAHAN

Nama Mata Kuliah	:algoritma dan struktur data
Judul Pratikum	:Function & array
Hari/Tanggal Pratikum	: 08 april 2026
Tanggal Penyerahan	:14 april 2026
Tempat	: Laboratorium (TIK.105)
Nama Mahasiswa	: Aulia illah
NIM	: 2025573010070
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

M.reza zulman ,S.ST.,M.Sc

1.1 Tujuan

1. Mendefinisikan dan memanggil function dengan parameter dan nilai kembalian (return value)
2. Memahami konsep scope variabel di dalam dan di luar function
3. Menulis arrow function sebagai sintaks modern pengganti function biasa
4. Memahami konsep callback dan penggunaannya sebagai argumen function
5. Membuat dan memanipulasi array menggunakan berbagai method bawaan
6. Menggunakan method iterasi array: forEach, map, filter, dan reduce
7. Memahami rekursi sebagai teknik function yang memanggil dirinya sendiri

1.2 langkah Langkah praktikum

berikut Langkah Langkah memulai praktikum:

1. Buka Git Bash, navigasikan ke root folder repository kamu.
2. Buat folder baru Bernama praktikum-3

```
mkdir praktikum-3
cd praktikum-3
```

3. Inisialisasi npm project.

```
npm init -y
```

4. Berikut stuktur praktikum-3

```
nama-repository/
└── praktikum-3/
    ├── package.json
    ├── 01-function-dasar.js
    ├── 02-arrow-callback.js
    ├── 03-array-dasar.js
    ├── 04-array-method.js
    ├── 05-rekursi.js
    └── tugas/
        ├── tugas-1.js
        └── tugas-2.js
```

A. Function

Konsep Function (Fungsi)

Function adalah blok kode yang diberi nama dan dapat dipanggil (dieksekusi) kapan saja dan berulang kali. Bayangkan function seperti sebuah mesin atau resep — kamu mendefinisikannya sekali, lalu bisa menggunakannya berkali-kali tanpa perlu menulis ulang kodenya.

Mengapa function penting? Prinsip utama pemrograman adalah DRY (Don't Repeat Yourself).Daripada menulis kode yang sama di banyak tempat, cukup bungkus kode itu dalam sebuahfunction.

Anatomi sebuah function:
• Parameter — Variabel input yang diterima function dari luar. Didefinisikan di dalam kurung ().
• Return value — Nilai yang dikembalikan function setelah selesai dieksekusi, menggunakan kata kunci return.
• Scope — Variabel yang dideklarasikan di dalam function hanya bisa diakses di dalam function itu saja (local scope). Variabel di luar function disebut global scope.

Perlu diingat: setelah JavaScript menemukan kata kunci return, eksekusi function langsung berhenti. Kode di bawah return tidak akan pernah dijalankan.

1) Ketik kode berikut di dalam terminal folder praktikum-3:

```
touch 01-function-dasar.js
```

2) Buka file 01-function-dasar.js di VS Code, lalu ketikkan kode berikut.

```
// 01-function-dasar.js
// ========================================
// FUNCTION:DEKLARASI,PRAMETER,RETURN,SCOPR
// ========================================

// --- 1. Function tanpa parameter dan tanpa return ---
function sapa() {
  console.log("Halo! Selamat datang di praktikum JavaScript.");
}
sapa(); // memanggil function
sapa(); // bisa dipanggil berkali-kali

// --- 2. Function dengan parameter ---
function sapaNama(nama) {
  console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama("Budi");
sapaNama("Siti");

// --- 3. Function dengan beberapa parameter dan return value ---
function tambah(angka1, angka2) {
  const hasil = angka1 + angka2;
  return hasil; // mengembalikan nilai
}
const hasilPenjumlahan = tambah(10, 25);
console.log("10 + 25 =", hasilPenjumlahan);
console.log("7 + 13 =", tambah(7, 13)); // bisa langsung dipakai

// --- 4. Function dengan default parameter ---
// Jika argumen tidak diberikan, gunakan nilai default
function hitung(nilai, pengali = 2) {
  return nilai * pengali;
```

```
}
console.log(hitung(5)); // 10 (pengali pakai default: 2)
console.log(hitung(5, 3)); // 15 (pengali diisi: 3)

// --- 5. Scope variabel ---
const pesanGlobal = "Saya ada di mana saja"; // global scope

function cekScope() {
  const pesanLokal = "Saya hanya ada di dalam function ini"; //
local scope
  console.log(pesanGlobal); // bisa akses global
  console.log(pesanLokal); // bisa akses lokal
}
cekScope();
```

3) ketik **node 01-function-dasar.js** .jika kode sama maka akan keluar hasil seperti berikut

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_
dat/praktikum-3 (main)
$ node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
```

B. Arrow Function & Callback

Arrow Function dan Konsep Callback

Arrow function adalah sintaks penulisan function yang lebih ringkas, diperkenalkan di ES6(2015). Alih-alih menulis function nama(params) { ... }, kita bisa menulis const nama = (params)=> { ... }.

Aturan penulisan arrow function:

- Jika hanya ada satu parameter, kurung bisa dihilangkan: x => x * 2
- Jika body hanya satu baris dan langsung return, kurung kurawal dan return bisa dihilangkan: (a, b) => a + b
- Jika body lebih dari satu baris, tetap gunakan kurung kurawal dan return.

Callback adalah sebuah function yang dikirimkan sebagai argumen ke function lain. Function penerima akan memanggil callback tersebut di waktu yang tepat. Ini adalah konsep yang sangat fundamental di JavaScript — hampir semua event handling, timer, dan method array menggunakan callback.
Analogi callback: Bayangkan kamu memesan makanan di restoran. Kamu

memberikan nomor mejamu (callback) ke pramusaji. Setelah makanan siap (event), pramusaji memanggil nomormejamu. Kamu tidak tahu kapan tepatnya dipanggil, tapi kamu sudah siapkan respon.

1. Buat file baru bernama 02-arrow-callback.js di dalam folder praktikum-3/.

```
touch 02-arrow-callback.js
```

2. Buka file 02-arrow-callback.js di VS Code, lalu ketikkan kode berikut.

```
// 02-arrow-callback.js
// ================================================
// ARROW FUNCTION DAN CALLBACK
// ================================================

// --- 1. Function biasa vs Arrow function ---

// Cara lama (function declaration)
function kuadratBiasa(x) {
  return x * x;
}

// Cara baru (arrow function - ekuivalen)
const kuadrat = (x) => {
  return x * x;
};

// Arrow function ringkas (implicit return)
const kuadratRingkas = (x) => x * x;

console.log("=== Perbandingan Penulisan ===");
console.log("Biasa   :", kuadratBiasa(5)); // 25
console.log("Arrow   :", kuadrat(5)); // 25
console.log("Ringkas :", kuadratRingkas(5)); // 25

// --- 2. Arrow function dengan beberapa parameter ---

const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;

console.log("Luas 4x6 :", luas(4, 6));
console.log(salam("Budi", "Pagi"));

// --- 3. Callback: function sebagai argumen ---

function lakukanOperasi(angka, operasiCallback) {
  console.log(`Angka awal: ${angka}`);
  const hasil = operasiCallback(angka);
  console.log(`Hasil setelah operasi: ${hasil}`);
}
```

```
console.log("\n=== Callback ===");
lakukanOperasi(7, kuadratRingkas); // kirim function sebagai argumen
lakukanOperasi(10, (x) => x + 100); // kirim arrow function langsung
lakukanOperasi(20, function (x) {
  // kirim function biasa langsung
  return x / 2;
});
// --- 4. setTimeout sebagai contoh callback nyata ---
console.log("\n=== setTimeout (Callback) ===");
console.log("Pesan 1: Sebelum timer");
setTimeout(() => {
  console.log("Pesan 3: Ini dari dalam setTimeout (setelah 1
detik)");
}, 1000); // 1000 milidetik = 1 detik
console.log("Pesan 2: Setelah mendaftarkan timer");
```

3.ketik node **02-arrow-callback.js** di terminal. Jika berhasil maka akan keluar hasil seperti berikut:

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_
dat/praktikum-3 (main)
$ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa   : 25
Arrow   : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
```

C. Array Dasar

Konsep Array

Array adalah struktur data yang menyimpan kumpulan nilai dalam satu variabel, berurutan dan terindeks. Bayangkan array seperti sebuah rak buku bernomor — setiap buku (elemen) memiliki posisi (indeks) yang dimulai dari angka 0.

Contoh: const buah = ['Apel', 'Mangga', 'Jeruk']
▪ buah[0] → 'Apel' (indeks pertama = 0)
▪ buah[1] → 'Mangga' (indeks kedua = 1)
▪ buah[2] → 'Jeruk' (indeks ketiga = 2)
▪ buah.length → 3 (jumlah elemen)

Array di JavaScript bersifat dinamis — ukurannya bisa berubah sewaktu-waktu. Array juga bisa menyimpan tipe data yang berbeda-beda dalam satu array (meskipun praktik terbaik adalah menyimpan satu jenis data per array).

Method bawaan array yang sering digunakan untuk manipulasi:
▪ push(x) — tambah elemen x di akhir array
▪ pop() — hapus dan kembalikan elemen terakhir
▪ unshift(x) — tambah elemen x di awal array
▪ shift() — hapus dan kembalikan elemen pertama
▪ splice(i, n) — hapus n elemen mulai indeks i
▪ slice(a, b) — salin elemen dari indeks a sampai sebelum b (non-destructive)
▪ indexOf(x) — cari indeks elemen x (-1 jika tidak ditemukan)
▪ includes(x) — cek apakah x ada di array (mengembalikan boolean)

1. Buat file baru bernama 03-array-dasar.js di dalam folder praktikum-3/.

```
touch 03-array-dasar.js
```

2. Buka file 03-array-dasar.js di VS Code, lalu ketikkan kode berikut.

```
// 03-array-dasar.js
// ================================================
// ARRAY: DEKLARASI, AKSES, DAN MANIPULASI
// ================================================
// --- 1. Membuat array ---
const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
const nilai = [85, 92, 78, 95, 88];
console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);
// --- 2. Mengakses elemen ---
console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]); // indeks 0
console.log("Elemen ketiga :", mahasiswa[2]); // indeks 2
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);
// --- 3. Mengubah elemen ---
mahasiswa[1] = "Siti Rahayu"; // ubah elemen indeks 1
console.log("\nSetelah diubah:", mahasiswa);
// --- 4. Menambah dan menghapus elemen ---
console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni"); // tambah di akhir
console.log("Setelah push :", mahasiswa);
mahasiswa.unshift("Zahra"); // tambah di awal
console.log("Setelah unshift :", mahasiswa);
```

```
const dihapusAkhir = mahasiswa.pop(); // hapus dari akhir
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);
const dihapusAwal = mahasiswa.shift(); // hapus dari awal
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);
// --- 5. Mencari elemen ---
console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));
// --- 6. Mengiris array dengan slice ---
const sebagian = nilai.slice(1, 4); // dari indeks 1 sampai sebelum
4
console.log("\nNilai asli :", nilai);
console.log("Slice [1,4] :", sebagian);
```

3. Jika hasilnya sesuai maka akan keluar seperti berikut:

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_
dat/praktikum-3 (main)
$ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]
```

D. Method Iterasi Array

Method Iterasi Array yang Modern

JavaScript menyediakan method iterasi array yang powerful. Method-method ini menerima sebuah callback function sebagai argumen, dan secara otomatis memanggil callback untuk setiap elemen array.

 `forEach(callback)` — Menjalankan callback untuk setiap elemen. Tidak mengembalikan nilai apapun. Digunakan hanya untuk efek samping (side effects) seperti mencetak atau menyimpan ke database.
 `map(callback)` — Membuat array baru dengan menerapkan callback ke setiap elemen. Array asli tidak berubah. Digunakan untuk mentransformasi setiap elemen.
 `filter(callback)` — Membuat array baru yang hanya berisi elemen yang memenuhi kondisi (callback mengembalikan true). Array asli tidak berubah. Digunakan untuk menyaring elemen.
 `reduce(callback, nilaiAwal)` — Mereduksi seluruh array menjadi satu nilai tunggal. Callback menerima dua argumen: accumulator (nilai yang terakumulasi) dan currentValue (elemen saat ini).
Ingat: map dan filter selalu menghasilkan array baru, sedangkan reduce menghasilkan satu nilai (angka, string, object, dll).

1. Buat file baru bernama 04-array-method.js di dalam folder praktikum-3/.

```
touch 04-array-method.js
```

2. Buka file 04-array-method.js di VS Code, lalu ketikkan kode berikut.

```
// 04-array-method.js
// ================================================
// METHOD ITERASI: forEach, map, filter, reduce
// ================================================
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
// --- 1. forEach: menjalankan sesuatu untuk setiap elemen ---
console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});
// --- 2. map: transformasi setiap elemen menjadi nilai baru ---
// Konversi nilai angka ke grade huruf
const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaiMahasiswa);
console.log("Grade :", gradeHuruf);
// --- 3. filter: saring elemen yang memenuhi kondisi ---
```

```
const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);
console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaiMahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai gagal :", nilaiGagal);
// --- 4. reduce: mereduksi array menjadi satu nilai ---
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0); // 0 adalah nilai awal akumulator
const rataRata = totalNilai / nilaiMahasiswa.length;
console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai :", totalNilai);
console.log("Rata-rata :", rataRata.toFixed(2));
// --- 5. Menggabungkan beberapa method (method chaining) ---
const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60) // ambil yang lulus dulu
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length; // bagi saat proses
  }, 0);
console.log("\n=== Method Chaining ===");
console.log("Rata-rata nilai lulus:",
rataRataNilaiLulus.toFixed(2));
```

3. Jika kode nya benar maka akan keluar hasil seperti berikut:

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struk
dat/praktikum-3 (main)
$ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]

=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75

=== Method Chaining ===
Rata-rata nilai lulus: 81.17
```

E. Rekursi

Konsep Rekursi (Recursion)

Rekursi adalah teknik di mana sebuah function memanggil dirinya sendiri untuk memecahkan masalah yang lebih kecil secara bertahap, sampai mencapai kondisi berhenti.

Setiap function rekursif harus memiliki dua bagian:
 Base case (kondisi berhenti) — kondisi di mana function tidak lagi memanggil dirinya sendiri. Tanpa ini, rekursi akan berjalan selamanya dan menyebabkan error Maximum call stack exceeded.
 Recursive case — bagian di mana function memanggil dirinya sendiri dengan input yang lebih kecil/sederhana.

Contoh: Menghitung faktorial dari n (n!):
- $5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! = 120$
- Base case: faktorial(1) = 1
- Recursive case: faktorial(n) = n × faktorial(n-1)

Rekursi sangat penting dalam Struktur Data dan Algoritma karena banyak algoritma pada pohon(tree), graf (graph), dan pencarian (searching) yang paling mudah diimplementasikan secara rekursif.

1. Buat file baru bernama 05-rekursi.js di dalam folder praktikum-3/.

```
touch 05-rekursi.js
```

2. Buka file 05-rekursi.js di VS Code, lalu ketikkan kode berikut.

```
// 05-rekursi.js
// ==================================
// REKURSI:BASE CASE DAN RECURSIVE CASE
// ====================================

// --- 1.faktorial:n! = n x (n-1)!---
function faktorial(n) {
  // Base case: berhenti saat n = 1
  if (n <= 1) return 1;

  // Recursive case: n × faktorial(n-1)
  return n * faktorial(n - 1);
}

console.log("=== Faktorial ===");
console.log("0! =", faktorial(0)); // 1 (konvensi matematika)
console.log("1! =", faktorial(1)); // 1
console.log("5! =", faktorial(5)); // 120
console.log("7! =", faktorial(7)); // 5040F

// --- 2. Fibonacci: F(n) = F(n-1) + F(n-2) ---
// Deret: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
function fibonacci(n) {
```

```
  // Base case
  if (n === 0) return 0;
  if (n === 1) return 1;
  // Recursive case
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log("\n=== Fibonacci ===");
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + " ");
}
console.log(""); // pindah baris
// --- 3. Rekursi untuk menelusuri array ---
function jumlahArray(arr, indeks = 0) {
  // Base case: sudah sampai akhir array
  if (indeks === arr.length) return 0;
  // Recursive case: elemen saat ini + jumlah sisa array
  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log("\n=== Jumlah Array dengan Rekursi ===");
console.log("Array :", angka);
console.log("Jumlah :", jumlahArray(angka)); // 26
// --- 4. Countdown rekursif ---
function countdown(n) {
  if (n < 0) {
    console.log("Selesai!");
    return;
  }
  console.log(n);
  countdown(n - 1);
}
console.log("\n=== Countdown Rekursif ===");
countdown(5);
```

3. Jika kodenya sesuai maka akan keluar hasil seperti berikut.

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur
dat/praktikum-3 (main)
$ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
```

F. Latihan

Latihan 1: Function Kalkulator Lengkap

1. Tambahkan kode baru di bawah kode yang sudah ada di file 01-function-dasar.js.
2. Buat function kurang(a, b) yang mengembalikan selisih a - b.
3. Buat function kali(a, b) yang mengembalikan hasil perkalian a * b.
4. Buat function bagi(a, b) yang mengembalikan hasil pembagian a / b. Tambahkan pengecekan: jika b === 0, tampilkan pesan 'Error: tidak bisa membagi dengan nol' dan return null.
5. Buat function kalkulator(a, operator, b) yang menerima 3 argumen. Gunakan if/else if atau switch untuk memanggil salah satu dari empat function di atas berdasarkan nilai operator ('+', '-', '*', '/').
6. Uji function kalkulator dengan setidaknya 5 pemanggilan berbeda, termasuk pembagian dengan nol.
7. Simpan dan jalankan: node 01-function-dasar.js.

Berikut kode Latihan :

```
console.log("\n\nlatihan");
// 2. Function kurang
function kurang(a, b) {
  return a - b;
}
```

```
// 3. Function kali
function kali(a, b) {
  return a * b;
}

// 4. Function bagi
function bagi(a, b) {
  if (b === 0) {
    console.log("Error: tidak bisa membagi dengan nol");
    return null;
  }
  return a / b;
}

// (Tambahan biasanya sudah ada: tambah)
// function tambah(a, b) {
//     return a + b;
// }

// 5. Function kalkulator
function kalkulator(a, operator, b) {
  switch (operator) {
    case "+":
      return tambah(a, b);
    case "-":
      return kurang(a, b);
    case "*":
      return kali(a, b);
    case "/":
      return bagi(a, b);
    default:
      console.log("Operator tidak dikenal");
      return null;
  }
}

// 6. Pengujian (minimal 5)
console.log('10 + 5 = ', kalkulator(10, "+", 5));
console.log('10 - 5 = ',kalkulator(10, "-", 5)); // 5
console.log('10 * 5 = ',kalkulator(10, "*", 5)); // 50
console.log('10 / 5 = ',kalkulator(10, "/", 2)); // 5
console.log('10 / 0 = ',kalkulator(10, "/", 0)); // Error
```

berikut hasilnya

```
latihan
10 + 5 =  15
10 - 5 =  5
10 * 5 =  50
10 / 5 =  5
Error: tidak bisa membagi dengan nol
10 / 0 =  null
```

Latihan 2: Pipeline Transformasi Data

1. Tambahkan kode baru di bawah kode yang sudah ada di file 02-arrow-callback.js.
2. Buat arrow function keHurufBesar yang menerima sebuah string dan mengembalikan string tersebut dalam huruf kapital semua. (Gunakan method .toUpperCase()).
3. Buat arrow function tambahSeru yang menerima sebuah string dan mengembalikan string tersebut dengan tanda !!! di akhir.
4. Buat arrow function hitungKata yang menerima sebuah string dan mengembalikan jumlah kata di dalamnya. (Petunjuk: gunakan .split(' ').length).
5. Buat function prosesKalimat(kalimat, transformasiCallback) yang menerima sebuah kalimat dan sebuah callback, lalu menerapkan callback ke kalimat tersebut dan menampilkan hasilnya.
6. Uji prosesKalimat dengan ketiga arrow function di atas menggunakan kalimat 'belajar javascript itu menyenangkan'.
7. Simpan dan jalankan. Pastikan outputnya sesuai harapan.

Berikut kode nya:

```javascript
// ================================================
// LATIHAN TAMBAHAN
// ================================================
// 1. Arrow function keHurufBesar
console.log("\n\nlatihan");

const keHurufBesar = (str) => str.toUpperCase();

// 2. Arrow function tambahSeru
const tambahSeru = (str) => str + "!!!";

// 3. Arrow function hitungKata
const hitungKata = (str) => str.split(" ").length;

// 4. Function dengan callback
function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);
  console.log(hasil);
}
```

```
// 5. Uji coba
const kalimat = "belajar javascript itu menyenangkan";

prosesKalimat(kalimat, keHurufBesar); // BELAJAR JAVASCRIPT ITU
MENYENANGKAN
prosesKalimat(kalimat, tambahSeru); // belajar javascript itu
menyenangkan!!!
prosesKalimat(kalimat, hitungKata); // 4
```

berikut hasilnya

```
latihan
BELAJAR JAVASCRIPT ITU MENYENANGKAN
belajar javascript itu menyenangkan!!!
4
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Latihan 3: Manajemen Daftar Belanja

1. Tambahkan kode baru di bawah kode yang sudah ada di file 03-array-dasar.js.
2. Buat array daftarBelanja berisi 5 item belanjaan.
3. Tampilkan seluruh isi daftar beserta nomor urutnya menggunakan for loop. Contoh output: 1. Beras, 2. Gula, dst.
4. Tambahkan 2 item baru di akhir daftar menggunakan push.
5. Hapus item pertama dari daftar menggunakan shift dan tampilkan item yang dihapus.
6. Cek apakah item 'Susu' ada di daftar menggunakan includes. Tampilkan hasilnya dengan keterangan yang jelas.
7. Tampilkan jumlah item akhir di daftar.

Berikut kodenya

```
console.log("\n\nlatihan");
// ================================================
// LATIHAN ARRAY DASAR
// ================================================

// 1. Membuat array daftar belanja
let daftarBelanja = ["Beras", "Gula", "Minyak", "Telur", "Garam"];

// 2. Menampilkan isi array dengan nomor urut
console.log("Daftar Belanja:");
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(`${i + 1}. ${daftarBelanja[i]}`);
}

// 3. Menambahkan 2 item baru
```

```
daftarBelanja.push("Susu", "Kopi");
console.log("\nSetelah ditambah item:");
console.log(daftarBelanja);

// 4. Menghapus item pertama
let itemDihapus = daftarBelanja.shift();
console.log(`\nItem yang dihapus: ${itemDihapus}`);

// 5. Cek apakah 'Susu' ada di daftar
let adaSusu = daftarBelanja.includes("Susu");
console.log(`Apakah ada 'Susu'? ${adaSusu ? "Ya, ada" : "Tidak
ada"}`);

// 6. Menampilkan jumlah item akhir
console.log(`Jumlah item akhir: ${daftarBelanja.length}`);
```

berikut hasilnya

```
latihan
Daftar Belanja:
1. Beras
2. Gula
3. Minyak
4. Telur
5. Garam

Setelah ditambah item:
[
  'Beras',  'Gula',
  'Minyak', 'Telur',
  'Garam',  'Susu',
  'Kopi'
]

Item yang dihapus: Beras
Apakah ada 'Susu'? Ya, ada
Jumlah item akhir: 6
```

Latihan 4: Analisis Data Produk
1. Tambahkan kode baru di bawah kode yang sudah ada di file 04-array-method.js. 2. Deklarasikan array of objects berikut: 3. const produk = [{ nama: 'Laptop', harga: 8500000, stok: 5 }, { nama: 'Mouse', harga:150000, stok: 0 }, { nama: 'Keyboard', harga: 450000, stok: 12 }, { nama: 'Monitor', harga: 3200000, stok: 0 }, { nama: 'Headset', harga: 350000, stok: 8 }] 4. Gunakan `filter` untuk mendapatkan array produk yang stok-nya lebih dari 0 (tersedia).Tampilkan hasilnya. 5. Gunakan `map` pada array produk yang tersedia untuk menghasilkan array baru berisi hanya nama produk-nya saja.

6. Gunakan `reduce` untuk menghitung total nilai inventaris dari semua produk yang tersedia (harga * stok).
7. Gunakan `forEach` untuk menampilkan semua produk dalam format: [TERSEDIA] Laptop -Rp8.500.000 (5 unit) atau [HABIS] Mouse - Rp150.000.
8. Simpan dan jalankan.

Berikut kodenya:

```
console.log("\n\nlatihan");
// ================================================
// LATIHAN ARRAY METHOD
// ================================================

// 1. Array of objects
const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

// 2. Filter produk yang tersedia (stok > 0)
const produkTersedia = produk.filter((item) => item.stok > 0);
console.log("Produk tersedia:");
console.log(produkTersedia);

// 3. Ambil hanya nama produk (map)
const namaProduk = produkTersedia.map((item) => item.nama);
console.log("\nNama produk tersedia:");
console.log(namaProduk);

// 4. Hitung total nilai inventaris (reduce)
const totalInventaris = produkTersedia.reduce(
  (total, item) => total + item.harga * item.stok,
  0,
);
console.log(`\nTotal nilai inventaris: Rp${totalInventaris}`);

// 5. Tampilkan semua produk (forEach)
console.log("\nDaftar semua produk:");
produk.forEach((item) => {
  const status = item.stok > 0 ? "[TERSEDIA]" : "[HABIS]";
  const hargaFormat = item.harga.toLocaleString("id-ID");

  if (item.stok > 0) {
    console.log(
      `${status} ${item.nama} - Rp${hargaFormat} (${item.stok}
unit)`,
    );
```

```
  } else {
    console.log(`${status} ${item.nama} - Rp${hargaFormat}`);
  }
});
```

Berikut hasilnya:

```
latihan
Produk tersedia:
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Nama produk tersedia:
[ 'Laptop', 'Keyboard', 'Headset' ]

Total nilai inventaris: Rp50700000

Daftar semua produk:
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000
[TERSEDIA] Headset - Rp350.000 (8 unit)
```

G. Tugas Praktikum

Tugas 1: Sistem Nilai Mahasiswa

1. Buka file tugas/tugas-1.js.
2. Deklarasikan array dataMahasiswa berisi minimal 6 object dengan struktur { nama: '...', nilai: ... }.
3. Buat function hitungStatistik(arrMahasiswa) yang menerima array mahasiswa dan mengembalikan sebuah object berisi: total (jumlah mahasiswa), rataRata (rata-rata nilai), tertinggi (nilai tertinggi), terendah (nilai terendah). Gunakan reduce untuk menghitungnya.
4. Buat function filterLulus(arrMahasiswa, batasLulus) yang menggunakan filter untuk mengembalikan array mahasiswa yang nilainya >= batasLulus.
5. Buat function tambahGrade(arrMahasiswa) yang menggunakan map untuk mengembalikan array baru dengan property tambahan grade (A/B/C/D/E) di setiap object.
6. Panggil semua function dan tampilkan hasilnya dengan format yang rapi menggunakan forEach dan template literal.

7. Simpan dan jalankan: node tugas/tugas-1.js.

Berikut kode tugas 1:

```
// Data mahasiswa (minimal 6)
const dataMahasiswa = [
  { nama: "Andi", nilai: 85 },
  { nama: "Budi", nilai: 70 },
  { nama: "Citra", nilai: 90 },
  { nama: "Dina", nilai: 60 },
  { nama: "Eka", nilai: 75 },
  { nama: "Fajar", nilai: 50 },
];

// Function hitung statistik
function hitungStatistik(arrMahasiswa) {
  const hasil = arrMahasiswa.reduce(
    (acc, mhs) => {
      acc.total += 1;
      acc.jumlahNilai += mhs.nilai;
      if (mhs.nilai > acc.tertinggi) acc.tertinggi = mhs.nilai;
      if (mhs.nilai < acc.terendah) acc.terendah = mhs.nilai;
      return acc;
    },
    {
      total: 0,
      jumlahNilai: 0,
      tertinggi: -Infinity,
      terendah: Infinity,
    },
  );

  return {
    total: hasil.total,
    rataRata: hasil.jumlahNilai / hasil.total,
    tertinggi: hasil.tertinggi,
    terendah: hasil.terendah,
  };
}

// Function filter lulus
function filterLulus(arrMahasiswa, batasLulus) {
  return arrMahasiswa.filter((mhs) => mhs.nilai >= batasLulus);
}

// Function tambah grade
function tambahGrade(arrMahasiswa) {
  return arrMahasiswa.map((mhs) => {
    let grade;
```

```
    if (mhs.nilai >= 85) grade = "A";
    else if (mhs.nilai >= 75) grade = "B";
    else if (mhs.nilai >= 65) grade = "C";
    else if (mhs.nilai >= 55) grade = "D";
    else grade = "E";

    return { ...mhs, grade };
  });
}

// Pemanggilan function
const statistik = hitungStatistik(dataMahasiswa);
const mahasiswaLulus = filterLulus(dataMahasiswa, 70);
const mahasiswaDenganGrade = tambahGrade(dataMahasiswa);

// Output statistik
console.log("=== Statistik Nilai Mahasiswa ===");
console.log(`Total Mahasiswa : ${statistik.total}`);
console.log(`Rata-rata Nilai : ${statistik.rataRata}`);
console.log(`Nilai Tertinggi : ${statistik.tertinggi}`);
console.log(`Nilai Terendah  : ${statistik.terendah}`);

// Output mahasiswa lulus
console.log("\n=== Mahasiswa Lulus (>= 70) ===");
mahasiswaLulus.forEach((mhs) => {
  console.log(`${mhs.nama} - ${mhs.nilai}`);
});

// Output semua mahasiswa dengan grade
console.log("\n=== Data Mahasiswa + Grade ===");
mahasiswaDenganGrade.forEach((mhs) => {
  console.log(`${mhs.nama} - Nilai: ${mhs.nilai}, Grade:
${mhs.grade}`);
});
```

Berikut hasil dari kode tersebut:

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-3/tugas (main)
$ node tugas-1.js
=== Statistik Nilai Mahasiswa ===
Total Mahasiswa : 6
Rata-rata Nilai : 71.66666666666667
Nilai Tertinggi : 90
Nilai Terendah  : 50

=== Mahasiswa Lulus (>= 70) ===
Andi - 85
Budi - 70
Citra - 90
Eka - 75

=== Data Mahasiswa + Grade ===
Andi - Nilai: 85, Grade: A
Budi - Nilai: 70, Grade: C
Citra - Nilai: 90, Grade: A
Dina - Nilai: 60, Grade: D
Eka - Nilai: 75, Grade: B
Fajar - Nilai: 50, Grade: E
```

Tugas 2: Pangkat dan Palindrom Rekursif

1. Buka file tugas/tugas-2.js.
2. Buat function rekursif pangkat(basis, eksponen) yang menghitung basis pangkat eksponen tanpa menggunakan operator `**. Gunakan rekursi: pangkat(b, e) = b × pangkat(b, e-1), dengan base case pangkat(b, 0) = 1`. Uji dengan beberapa nilai.
3. Buat function rekursif balikString(str) yang membalik sebuah string. Contoh: 'halo' → 'olah'.Base case: string kosong atau satu karakter. Recursive case: karakter terakhir + balikString(sisa string). Petunjuk: gunakan str[str.length - 1] dan str.slice(0, str.length - 1).
4. Buat function cekPalindrom(str) yang memanfaatkan balikString untuk mengecek apakah sebuah string adalah palindrom (sama jika dibaca terbalik). Contoh palindrom: 'katak','civic', 'level'. Uji dengan minimal 4 kata.
5. Simpan dan jalankan: node tugas/tugas-2.js.

Berikut kode tugasnya:

```javascript
// Function rekursif pangkat
function pangkat(basis, eksponen) {
  if (eksponen === 0) return 1; // base case
  return basis * pangkat(basis, eksponen - 1);
}

// Uji pangkat
console.log("=== Uji Pangkat ===");
console.log(`2^3 = ${pangkat(2, 3)}`);
console.log(`5^2 = ${pangkat(5, 2)}`);
console.log(`3^4 = ${pangkat(3, 4)}`);
```

```
console.log(`10^0 = ${pangkat(10, 0)}`);

// Function rekursif balik string
function balikString(str) {
  if (str.length <= 1) return str; // base case
  return str[str.length - 1] + balikString(str.slice(0, str.length -
1));
}

// Uji balik string
console.log("\n=== Uji Balik String ===");
console.log(`halo -> ${balikString("halo")}`);
console.log(`javascript -> ${balikString("javascript")}`);

// Function cek palindrom
function cekPalindrom(str) {
  const dibalik = balikString(str);
  return str === dibalik;
}

// Uji palindrom
console.log("\n=== Uji Palindrom ===");
const kata = ["katak", "civic", "level", "halo"];

kata.forEach((k) => {
  if (cekPalindrom(k)) {
    console.log(`${k} adalah palindrom`);
  } else {
    console.log(`${k} bukan palindrom`);
  }
});
```

Berikut hasil dari kodenya:

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-3/tugas (main)
$ node tugas-2.js
=== Uji Pangkat ===
2^3 = 8
5^2 = 25
3^4 = 81
10^0 = 1

=== Uji Balik String ===
halo -> olah
javascript -> tpircsavaj

=== Uji Palindrom ===
katak adalah palindrom
civic adalah palindrom
level adalah palindrom
halo bukan palindrom
```

H. Submit ke GitHub

Kembali ke root folder repository.

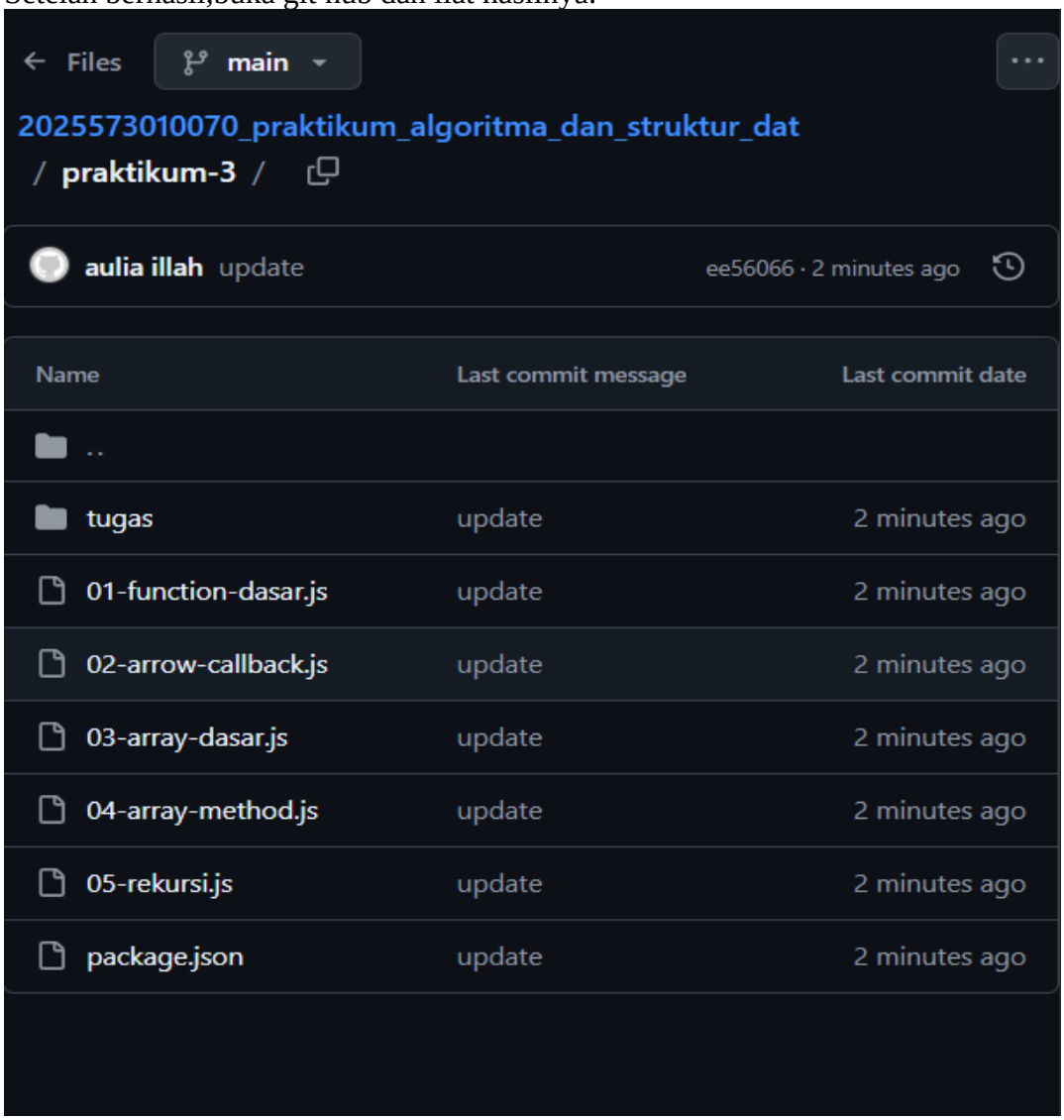
```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat (main)
$ cd praktikum-3

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-3 (main)
$ git add .
warning: in the working copy of 'praktikum-3/package.json', LF will be replaced by CRLF the next time Git touches it

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-3 (main)
$ git commit -m "update"
[main ee56066] update
 8 files changed, 525 insertions(+)
 create mode 100644 praktikum-3/01-function-dasar.js
 create mode 100644 praktikum-3/02-arrow-callback.js
 create mode 100644 praktikum-3/03-array-dasar.js
 create mode 100644 praktikum-3/04-array-method.js
 create mode 100644 praktikum-3/05-rekursi.js
 create mode 100644 praktikum-3/package.json
 create mode 100644 praktikum-3/tugas/tugas-1.js
 create mode 100644 praktikum-3/tugas/tugas-2.js

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-3 (main)
$ git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.52 KiB | 667.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/auliaillah-cell/2025573010070_praktikum_algoritma_dan_struktur_dat.git
   4763bb5..ee56066  main -> main
```

Setelah berhasil,buka git hub dan liat hasilnya:



Kesimpulan:

Dalam praktikum ini, dipelajari berbagai konsep dasar hingga lanjutan dalam pemrograman JavaScript yang berfokus pada penggunaan function dan pengolahan data. Mahasiswa memahami cara mendefinisikan dan memanggil function dengan parameter serta nilai kembalian, sehingga program dapat dibuat lebih modular dan fleksibel. Selain itu, konsep scope membantu dalam mengelola akses variabel agar tidak terjadi konflik data.

Penggunaan arrow function diperkenalkan sebagai sintaks modern yang lebih ringkas dibandingkan function biasa. Selanjutnya, konsep callback memberikan pemahaman tentang bagaimana function dapat digunakan sebagai argumen untuk meningkatkan fleksibilitas dan interaktivitas program.

Dalam pengolahan data, array menjadi struktur penting yang dapat dimanipulasi menggunakan berbagai method bawaan seperti forEach, map, filter, dan reduce untuk melakukan iterasi dan transformasi data secara efisien. Terakhir, rekursi dipahami sebagai teknik pemanggilan function terhadap dirinya sendiri untuk menyelesaikan masalah tertentu secara lebih elegan.

Secara keseluruhan, praktikum ini meningkatkan pemahaman dalam menulis kode JavaScript yang lebih terstruktur, efisien, dan modern.